

Sécurité avancée des SI d'entreprise

OS 101

Daniel De Almeida Braga

What's an OS?



A wide variety of choice



A wide variety of choice



All different, but still the same

 **Operating system:** System software that manages computer hardware and software resources, and provides common services for computer programs.

- Different flavor are called *distributions* (e.g., Ubuntu, Debian, ...)
- Different OS tackle issues differently, but core concepts are often similar
- Overall, most of them have a common goal

What's their purposes?

1. Resources management (allocation between user program, file system, i/o devices, ...)
2. Abstract complex details of the hardware and provide an easy environment for the user

What's their purposes?

1. **Efficient** resources management (allocation between user program, file system, i/o devices, ...)
2. Abstract complex details of the hardware and provide an easy environment for the user

What's their purposes?

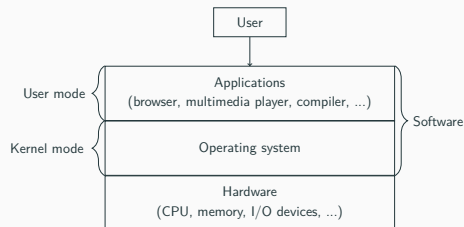
1. **Efficient** resources management (allocation between user program, file system, i/o devices, ...)
2. Abstract complex details of the hardware and provide an easy environment for the user

Its goals are to be:

- User friendly
- Efficient (time, space and energy)
- Reliable
- **Secure and protected**

OS abstraction

- Abstraction turns a difficult task into 2 manageable ones
- Programmers use API to do complex things
- Critical operations are deferred to the OS
- OS manages multiplexing



I don't expect you to know everything, but to be able to
figure thing out

I don't expect you to know everything, but to be able to
figure thing out

RTFM: Read The F*cking Manual

Let's talk security

At a high level

- How do we authenticate users?

At a high level

- How do we authenticate users?
- How do we make sure that anybody cannot access anything?

At a high level

- How do we authenticate users?
- How do we make sure that anybody cannot access anything?
- How do we make sure that programs can't run whatever they want?

At a high level

- How do we authenticate users?
- How do we make sure that anybody cannot access anything?
- How do we make sure that programs can't run whatever they want?
- How do we isolate processes from one another?

At a high level

- How do we authenticate users?
- How do we make sure that anybody cannot access anything?
- How do we make sure that programs can't run whatever they want?
- How do we isolate processes from one another?
- How can we protect data (at rest, or not)?

Trusted Computing Base

You need to **trust** something: the set of hardware, firmware and software that should ensure the security policy: the **Trusted Computing Base (TCB)**.

- An issue in TCB can compromise the entire system security

The TCB needs to be as small as possible, to ensure easy security audit, enable formal verification, ...

- Ideally: hardware + subset of the kernel

Theory vs Practice



Theory vs Practice

- All the kernel...including third party drivers
- Some userspace services

On Linux

- Login and authentication binaries
- Recent kernels are < 30M lines of code

On Windows

- LSASS service (authentication)
- Windows kernel was < 45M lines of code...in 2011

Theory vs Practice

- All the kernel...including third party drivers
- Some userspace services

On Linux

- Login and authentication binaries
- Recent kernels are < 30M lines of code

On Windows

- LSASS service (authentication)
- Windows kernel was < 45M lines of code...in 2011

Trusted != Trustworthy != Secure

Trusted Execution Environments (TEE)

TEE enable anti-tampering of (part of) the TCB

Security certifications

Certifications

Certification can bring a sense of security (⚠ rarely a proof of security)

CSPN

- Certification de Sécurité de Premier Niveau
- Delivered by the ANSSI **for a specific software version**
- Shallow evaluation of the security
- Recognized in France and Germany (agreement with BSI)

Common Criteria

- Multi level certification (7 levels)
- Worldwide recognition (31 countries)
- 18 countries can deliver the certifications (5 lab in France)
- Usually much longer (and expensive) process

CC is not always better than CSPN!

Common criteria levels

The higher the level is, the more verification was done

- EAL1: Functionally Tested
- EAL2: Structurally Tested
- EAL3: Methodically Tested and Checked
- EAL4: Methodically Designed, Tested and Reviewed
- EAL5: Semi-Formally Designed and Tested
- EAL6: Semi-Formally Verified Design and Tested
- EAL7: Formally Verified Design and Tested

Don't be a fool

- Evaluation are usually focused on *a part* of the product: the Target of Evaluation (ToE)
 - Other elements are considered out of scope
- $EAL < 3$ don't check for security
- Even certified product can have vulnerabilities

Get around your OS

The case of Linux

Shell basics

- Shells are command-line interfaces for Unix-like OS
- We can type command, and use a scripting language to automate repetitive tasks
- Many different shells, which vary slightly
 - sh (Bourne)
 - bash (Bourne Again)
 - zsh (Z shell)
 - ...
- Documentation is available using `man`

Command line structure

- `$ command -o -long-option argument`
- “command” are actually binaries, stored somewhere in your PATH
- PATH is a list of directory to look for commands

Default PATH usually contains `/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin`

- There are multiple ways to find where a command exists in the PATH
 - `which` locates a command (shell built-in)
 - `whereis` locate the binary, source, and manual page files for a command

Redirection

3 data channels:

0 `stdin` - input (e.g., keyboard)

1 `stdout` - output (e.g., screen)

2 `stderr` - output error (e.g., screen)

Redirection

3 data channels:

0 `stdin` - input (e.g., keyboard)

1 `stdout` - output (e.g., screen)

2 `stderr` - output error (e.g., screen)

You can redirect channels to files, or to each other using `>`

- Redirect `stdout` to `file.log`: `cmd > file.log`
- Append `stdout` to `file.log`: `cmd » file.log`
- Redirect errors to `/dev/null`: `cmd 2> /dev/null`
- `cmd 2>&1 » file.log`

Redirection

3 data channels:

0 `stdin` - input (e.g., keyboard)

1 `stdout` - output (e.g., screen)

2 `stderr` - output error (e.g., screen)

You can redirect channels to files, or to each other using `>`

- Redirect stdout to file.log: `cmd > file.log`
- Append stdout to file.log: `cmd » file.log`
- Redirect errors to `/dev/null`: `cmd 2> /dev/null`
- Redirect errors to stdout, then append stdout to file.log: `cmd 2>&1 » file.log`

Pipes

Pipes (|) takes the output of a command, and give it as input to the next.

This is very useful to chain commands

- List all users alphabetically: `cat /etc/passwd | awk -F':' '{print $1}' | sort`
- Find processes containing a string: `ps aux | grep docker`

Script

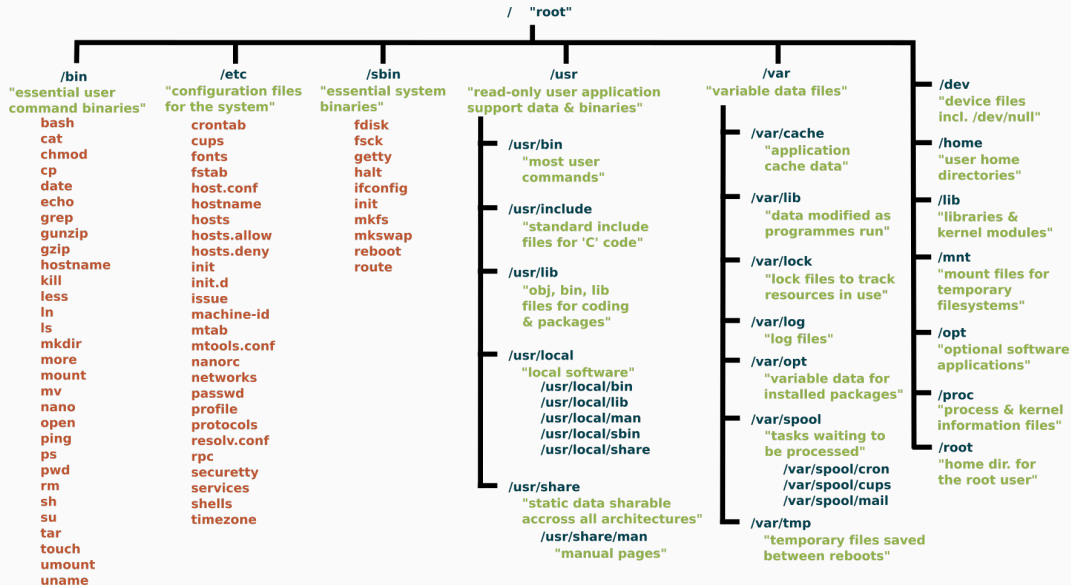
Chain of commands stored in a file, that will be executed linearly

- The file needs execution right (`chmod +x script`)
- Can be executed using its path (`./script`) or by placing it in a folder of your `PATH`
- Scripts must start with a shebang (`#!`) followed by the interpreter
 - `#!/bin/bash` for instance

Everything is a file

- Files have an owner, and related access permissions
- Directories are files listing locations
- System expose many things (config, ...) through the filesystem
- /proc/ contains many process-related info exposed through files
- Many basic commands are text manipulation tools (grep, awk, sed, ...) and can be used for generic and administration tasks alike

Linux filesystem



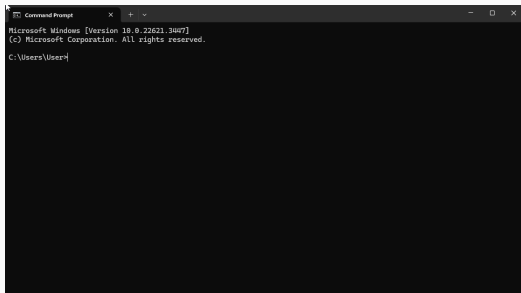
Get around your OS

The case of Windows

CMD vs PowerShell

CMD:

- CMD is legacy, (MS-DOS systems)



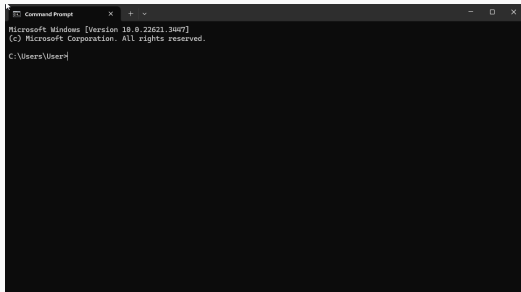
PowerShell:

- Shell *and* a scripting language
- More flexible and powerful
- Designed for system administration

CMD vs PowerShell

CMD:

- CMD is legacy, (MS-DOS systems)



```
Microsoft Windows [Version 10.0.22621.3447]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>
```

PowerShell:

- PowerShell is blue
- Shell *and* a scripting language
- More flexible and powerful
- Designed for system administration

```
PS C:\> Get-Childitem 'MediaCenter:\Music' -rec |
>> where < -not $_.PSIsContainer -and $_.Extension -match 'mainp3' > |
>> Measure-Object -property length -sum -min -max -ave
>>

Count       : 1307
Average     : 5491276.09563887
Sum         : 7177097857
Maximum     : 22905267
Minimum     : 3235
Property    : Length

PS C:\> Get-WmiObject CIM_BIOSElement | select biosv*, man*, ser* | Format-List

BIOSVersion : <TOSCPL - 6040000, Ver 1.00PARTIBL>
Manufacturer : TOSHIBA
SerialNumber : M821116H
```

Why PowerShell?

Unix-shell does the job very well: why not using the same on Windows?

- Linux as a document oriented design (everything is a file)
- Windows has an API oriented design

We can `awk` or `grep` in the registry

We need a way to administrate using the API: PowerShell

Powershell 101

- Extensive command set (cmdlets)
- Scripts use .ps1 extensions
- Most administration tasks need an admin PowerShell

cmdlets structure: Verb-Noun -Option
argument

- `cat` $\Rightarrow$ `Get-Content`
- `cd` $\Rightarrow$ `Set-Location`
- `ls/dir` $\Rightarrow$ `Get-Childitem`
- `cp/copy` $\Rightarrow$ `Copy-Item`

Native commands (`ping`, `calc`, ...) also work

Baby steps: you can use aliases to get started

- Try running `get-alias`

Redirections

Very similar to shell...with more channels and cmdlets!

1. Success Stream (`Write-Output`)
2. Error Stream (`Write-Error`)
3. Warning Stream (`Write-Warning`)
4. Verbose Stream (`Write-Verbose`)
5. Debug Stream (`Write-Debug`)
6. Information Stream (`Write-Information`, `Write-Host`)

Redirections

Very similar to shell...with more channels and cmdlets!

1. Success Stream (Write-Output)
2. Error Stream (Write-Error)
3. Warning Stream (Write-Warning)
4. Verbose Stream (Write-Verbose)
5. Debug Stream (Write-Debug)
6. Information Stream (Write-Information, Write-Host)

Redirection operators are mostly to redirect one stream to another:

- `.\script.ps1 > script.log`
- `.\script.ps1 3>&1 2>&1 > script.log`
- `.\script.ps1 *> script.log`

Pipes

Same concept as for shell: chaining command by redirecting output to input... but it's not only text but objects!

- `Get-Process notepad | Stop-Process`
 - The process is sent through the pipeline
- `Get-Item -Path HKLM:\Software\MyCompany | New-ItemProperty -Name nEmployees -Value 98`
- ■ Adds a new register entry

Pipes

Same concept as for shell: chaining command by redirecting output to input... but it's not only text but objects!

- `Get-Process notepad | Stop-Process`
 - The process is sent through the pipeline
- `Get-Item -Path HKLM:\Software\MyCompany | New-ItemProperty -Name nEmployees -Value 98`
- ■ Adds a new register entry

You can see which options of a cmdlets accepts pipeline input using

- `Get-Help <cmdlet> -Full`
- `Get-Help <cmdlet> -Parameter *`

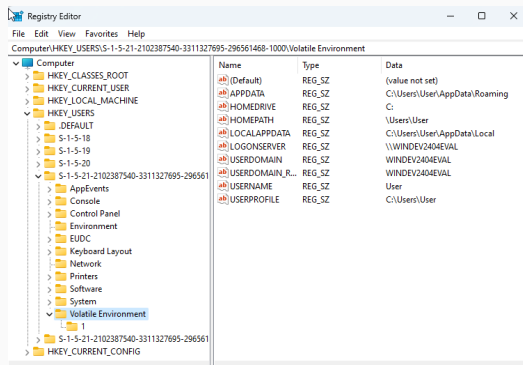
Windows registry

 **Windows Registry:** Hierarchical database that stores low-level settings for the Microsoft Windows operating system and for applications that opt to use the registry.

Windows registry

 **Windows Registry:** Hierarchical database that stores low-level settings for the Microsoft Windows operating system and for applications that opt to use the registry.

- Tree structure
- Basic elements: keys and value
- A key may contain sub-keys and/or values
- Can be read and modified within regedit



Predefined root key

- HKEY_LOCAL_MACHINE (HKLM): System-related information
- HKEY_CURRENT_CONFIG (HKCC): Current hardware profile
- HKEY_CLASSES_ROOT (HKCR): Information about registered apps
- HKEY_CURRENT_USER (HKCU): Settings of the logged-on user
- HKEY_USERS (HKU): Settings for each user in the system
- HKEY_PERFORMANCE_DATA (invisible): Performance information
- HKEY_DYN_DATA (only Windows 9x)

Predefined root key

- `HKEY_LOCAL_MACHINE` (HKLM): System-related information
- `HKEY_CURRENT_CONFIG` (HKCC): Current hardware profile
- `HKEY_CLASSES_ROOT` (HKCR): Information about registered apps
- `HKEY_CURRENT_USER` (HKCU): Settings of the logged-on user
- `HKEY_USERS` (HKU): Settings for each user in the system
- `HKEY_PERFORMANCE_DATA` (invisible): Performance information
- `HKEY_DYN_DATA` (only Windows 9x)

Registry Values

- Pair of name and data stored in keys
- Unique name within a key
- Can have different types
 - string (REG_SZ, REG_EXPAND_SZ)
 - binary (REG_BINARY)
 - word (REG_DWORD, REG_QWORD)
 - link (REG_LINK)
 - resource list (REG_RESOURCE_LIST)
 - resource descriptor (REG_RESOURCE_DESCRIPTOR)
 - resource requirements (REG_RESOURCE_REQUIREMENTS)